

MCU 中断、七段数码管与矩阵键盘的编程实践

指导老师：李玉虎

引言

此主要围绕 ATmega8A 微控制器（MCU）的中断机制、七段数码管显示及矩阵键盘的编程进行实践。通过这些内容，我不仅能够深入理解 MCU 的工作原理，还能掌握如何通过编程实现具体的硬件控制任务。本此则重点介绍了 MCU 的中断机制及其编程方法。

一. 中断

1.1 中断基本概念

允许/开中断/中断使能：通过设置 SREG 寄存器中的 I 位来允许全局中断或特定中断（如 INTO 或 INT1）。

中断向量表：每个中断源都有一个对应的入口地址，用于跳转到中断服务程序（ISR）。

中断服务程序：类似于函数，用于处理特定中断事件。

中断现场的保护与恢复：在进入 ISR 之前保存断点数据，处理完成后恢复现场。

禁止中断：通过清零 SREG 寄存器中的 I 位来关闭全局中断。

1.2 中断过程

当常规任务执行时，如果出现中断请求，CPU 会跳转到中断向量表中对应中断的入口地址处执行。

在 ISR 中，首先需要关中断以保护现场，然后处理特定任务，最后恢复现场并开中断。

1.3 外部中断的开与关

通用中断控制寄存器(GICR)：用于控制 INTO 和 INT1 的中断使能。

控制寄存器(MCUCR)：用于设置外部中断的触发方式（低电平、上升沿、下降沿或任意电平变化）。

1.4 实例分析

通过一个实例，学习了如何使用中断统计触摸开关的开/关次数具体步骤包括设置中断触发方式、允许中断、编写 ISR 以及主循环中的处理逻辑。

二. 七段数码管显示

2.1 一位七段数码管

一位七段数码管由 a 到 g 七个段和一个公共端组成，共阴极或共阳极连接方式不同。

通过控制相应 IO 端口的高低电平，可以点亮不同的段组合以显示数字。

2.2 多位七段数码管

多位七段数码管由多个一位数码管组成，通常采用动态扫描方式显示。

动态扫描通过快速切换每一位的显示状态，利用人眼的视觉暂留效应实现多位同时显示。

2.3 实例分析

通过实例，学习了如何用一位和四位七段数码管分别显示触摸开关的开/关次数。

具体步骤包括设置 IO 端口模式、编写显示函数以及在主循环中调用该函数。

矩阵键盘

矩阵键盘：矩阵键盘是一种常用的输入设备，通过行列交叉形成按键阵列来实现多键输入。

在本节中，学习了如何通过 MCU 读取矩阵键盘的状态。

3.1 矩阵键盘结构

矩阵键盘由行和列组成，行连接 MCU 的输出端口，列连接 MCU 的输入端口。

通过扫描行和列的状态，可以判断哪个按键被按下。

3.2 读行和读列

读行：向行管脚发送低电平，读取列管脚的状态。

读列：向列管脚发送高电平，读取行管脚的状态。

3.3 实例分析

通过实例，我们学习了如何将矩阵键盘最近四次按下的键编码并显示在数码管上。

具体步骤包括设置 IO 端口模式、编写扫描函数以及在主循环中调用该函数。

四. 思考与技术分析：

在实验中需要注意的一些问题：

1. 实验内容 1：将 INT1 管脚上连接的触摸开关的开关次数显示在一位数码管上：

在这个过程中要将 INTO 改为 INT1，在这里要注意不仅仅要修改代码，同时还要修改硬件的连接方式。

关键修改：

输入引脚设置：

修改 DDRD 中 INTO 相关配置为 INT1。INT1 连接到 PD3 引脚，因此 `DDRD &= ~(1 << DDD3)` 将 PD3 设置为输入。

中断触发设置：

将 MCUCR 寄存器的 ISC01 和 ISC00 修改为 ISC11 和 ISC10，这使得 INT1 在上升沿触发中断。

中断向量修改：

将中断服务程序的名字由 INTO_vect 改为 INT1_vect，以响应 INT1 的中断。

```
int main(void)
{
    unsigned char seg7_hex[16] =
    {0xfc, 0x60, 0xda, 0xf2, 0x66, 0xb6, 0xbe, 0xe6,
     0xfe, 0xf6, 0xee, 0x3e, 0x9c, 0x7a, 0x9e, 0x8e};

    DDRB = 0xff; // PB7~0 为输出，分别控制七段数码管的 a~g 段
    DDRC = 0x01; // PC0 为输出，控制数码管的共阴极管脚
    PORTC = 0x00; // PC0 输出低电平 '0'

    // 配置 INT1 中断
    DDRD &= ~(1 << DDD3); // PD3 (INT1) 为输入 (从 1 开始)
    MCUCR |= (1 << ISC11) | (1 << ISC10); // INT1 上升沿触发中断
    GICR |= (1 << INT1); // 开启 INT1 中断
    sei(); // 开启全局中断 SREG (I)
```

实验内容 2：将 INT1 管脚上的触摸开关的开关次数显示在四位共阴极七段数码管上。

1. 引脚配置：

`DDRD &= ~(1 << DDD3)`; 将 PD3 引脚配置为输入，以便连接触摸开关。

2. 中断配置：

`MCUCR |= (1 << ISC11) | (1 << ISC10)`; 将 INT1 设置为上升沿触发中断。

3. 中断使能：

`GICR |= (1 << INT1)`; 启用 INT1 外部中断。

4.中断服务程序:

ISR(INT1_vect): 这是处理 INT1 中断的函数。

这样修改后, 程序会响应 PD3 引脚上的上升沿中断, 也就是当触摸开关触发时, 计数器会递增, 数码管显示的数字会随之增加。

```
DDRC = (0x0f);
DDRB = (0xff);
DDRD &= ~(1<<DDRD3);
MCUCR |= ((1<<ISC11)|(1<<ISC10));
GICR |= (1<<INT1);
sei();
while (1)
{
    d_t = counter;
    for(i=0;i<4;i++)
    { PORTC |=0x0f; //禁止显示
      PORTB =seg7_hex[d_t%10]; //送数
      PORTC =seg7_com[i]; //显示
      d_t=d_t/10; //下一位
      _delay_ms(5);
    }
}
ISR(INT1_vect)
{if(counter < 9999) counter++;
 else counter = 0; }
```

实验内容 3: 根据小键盘示例和拓展知识等, 真正实现将小键盘最近四次按下键的编码, 按顺序编码并显示在数码管上。

按键输入和显示重复输入现象:

这个问题通常由按键按下时间和 MCU 的处理时间不匹配导致。你提到的重复输入现象可能是因为按键被多次扫描或者按键未被消抖处理。可以通过延时控制和改进按键扫描策略来避免这个问题。

减少 MCU 处理时间:

在按键扫描后可以增加一些延时 (比如 50-100ms), 这有助于避免重复读取同一个按键的多次状态。

```
if(keyno<16) /*扫描一轮并编码后*/
{ seg7_no[3]=seg7_no[2]; //移动按键数据
  _delay_ms(500);
  seg7_no[2]=seg7_no[1];
  _delay_ms(500);
  seg7_no[1]=seg7_no[0];
  _delay_ms(500);
  seg7_no[0]=keyno; //新的按键数据
  _delay_ms(500);
}
```

五.结论

通过本次课程的实践, 我不仅掌握了 MCU 中断机制、七段数码管显示及矩阵键盘的编程方法, 还学会了如何在实际项目中应用这些技术。在未来的学习和工作中, 我将继续深入研究这些技术, 并结合实际项目进行优化和改进。